

CS633: Parallel Computing

Assignment-2 Report

Group Number 61

Aayush Kumar (230027) Aritra Ambudh Dutta (230191) Cezan Vispi Damania (230310)
Suvid Goel (231065) Swapnil Garg (231069)

Date: April 12, 2026

1 Code Description

Our program performs a **3D stencil computation** on a domain distributed across a $p_x \times p_y \times p_z$ process grid, where each process owns a local sub-domain of size $n_x \times n_y \times n_z$. Every process maintains a padded buffer of dimensions $(n_x+2h) \times (n_y+2h) \times (n_z+2h)$, where the halo depth $h = \lfloor (d-1)/6 \rfloor$ is derived from the stencil parameter d . The ghost cells around the sub-domain hold data imported from neighbouring processes during each iteration.

For T iterations, each process exchanges halo layers with its six face-neighbours, applies a d -point weighted-average stencil, and then counts **isovalue crossings** across edges of the local grid. After all iterations, Rank 0 collects global crossing counts via `MPI_Reduce` and reports timing.

1.1 Modular Decomposition

The implementation is organized into 3 phases: one-time setup, the iterative core, and final aggregation.

Phase 1: Initialization and Setup

- **Argument Parsing & MPI Setup:** The program reads 12 command-line arguments ($d, p_n, p_x, p_y, p_z, n_x, n_y, n_z, T, \text{seed}, F, \text{isovalue}$) and initializes MPI. Two arrays of F field pointers — `data` and `n_data` — are allocated as padded 3D buffers of $(n_x+2h)(n_y+2h)(n_z+2h)$ doubles each, constituting a double-buffering scheme: the stencil kernel reads from one buffer and writes to the other, after which the pointers are exchanged.
- **Data Initialization:** After seeding via `srand(seed)`, the `initialize()` function fills only the interior cells (index range $[h, n_x+h)$ per dimension) following the assignment specification. Ghost cells remain uninitialised at this stage; they are populated by the first halo exchange.
- **Neighbour Ranks & MPI Derived Datatypes:** Each process determines the MPI ranks of its six face-neighbours from the Cartesian topology (marking absent neighbours as -1). Six `MPI_Type_vector` descriptors are then registered — three for the stencil halo (thickness h) and three for the isocount halo (thickness 1), enabling `MPI_Isend/MPI_Irecv` to address non-contiguous slabs of the padded array directly without manual pack/unpack.

Phase 2: The Iterative Core (T iterations)

Each iteration comprises three stages executed in strict sequence.

- **Stage 1: Overlapped Halo Exchange and Interior Stencil:** All six halo exchanges are posted as non-blocking `MPI_Isend/MPI_Irecv` calls, after which `stencil_compute_core()` immediately evaluates the stencil for interior points (index range $[2h, n_x)$ in each dimension), which have no dependency on ghost data and thus execute concurrently with the transfers. Upon `MPI_Waitall()` completion, the boundary stencil proceeds.
- **Stage 2: Boundary Stencil Computation:** `stencil_compute_boundaries()` processes six boundary shells in three non-overlapping pairs (Front/Back, North/South, East/West). At each point, a running count `nval` admits a direction only if the index lies within the owned domain or a valid MPI neighbour exists, and the average divides by `nval` rather than d for correctness at faces, edges, and corners. The `data/n_data` pointers are then swapped.
- **Stage 3: Isovalue Crossing Count:** A second non-blocking exchange posts 1-layer iso halos for all six directions. After `MPI_Waitall`, `iso_compute()` checks only the three negative directions ($-x, -y, -z$) per grid data point, visiting each undirected edge exactly once. The given isovalue on an edge (P, Q) exists if

$(\text{isovalue} - v_P)(\text{isovalue} - v_Q) \leq 0$. At process boundaries both adjacent processes count it via the ghost layer; at domain boundaries the check is skipped.

Phase 3: Final Aggregation

A single `MPI_Reduce` with `MPI_SUM` merges all per-process `isocount` arrays (size $F \times T$) into `global_count` at Rank 0, which prints T lines of F counts each. Wall-clock time is measured with `MPI_Wtime()` around the entire loop, and a second `MPI_Reduce` with `MPI_MAX` extracts the bottleneck process time, which Rank 0 appends as the final output line.

1.2 Key Optimizations

1. **Communication–Computation Overlap:** All six halo exchanges are posted as non-blocking `MPI_Isend/MPI_Irecv` before any arithmetic begins. While the transfers are in flight, `stencil_compute_core()` processes the interior — points at least h away from every face, whose stencil footprint never touches ghost cells. Only after `MPI_Waitall` confirms completion do the boundary shells run. For $d=7$ on a 120^3 sub-domain the interior is $118^3 \approx 95.08\%$ of all points, so nearly the entire stencil executes concurrently with the data transfer, converting a serial “communicate then compute” sequence into an overlapped pipeline and effectively hiding the halo transfer latency behind productive arithmetic.
2. **Three-Direction Isocount:** Instead of sweeping all 26 neighbours per cell and maintaining canonical-direction bookkeeping to prevent double-counting, `iso_compute()` examines only the three negative directions ($-x, -y, -z$). This ensures each undirected interior edge is visited exactly once, since the cell with the higher coordinate implicitly owns the edge. At process boundaries, this negative direction sweep naturally prevents inter-process double counting. The process with the higher rank evaluates the shared boundary edge by looking into its ghost later, while the neighboring lower rank process looks into its own local domain only. Consequently, every edge spanning across sub-domains is evaluated by exactly one process, ensuring the global `MPI_Reduce` sums only unique, non-duplicated counts. When no neighbour exists in a given direction (i.e. a domain boundary), a hoisted boolean flag causes the outermost face layer to bypass that check entirely. The crossing test itself employs the branchless product form $(\text{iso} - v_1)(\text{iso} - v_2) \leq 0$ — a single multiply followed by a sign test — which is readily amenable to auto-vectorisation by the compiler.
3. **MPI Derived Datatypes (Zero-Copy Halo):** Face slabs of the padded 3D buffer are non-contiguous in memory — e.g. a North–South slab comprises multiple short strips separated by a full-plane stride. `MPI_Type_vector` descriptors encoding each face’s stride pattern are registered, enabling the MPI runtime to transfer data directly from and into the padded array without requiring any manual pack/unpack buffer. Six types are committed once at startup (three for the stencil halo of thickness h , three for the iso halo of thickness 1) and reused across all $T \times F$ exchanges, eliminating temporary buffers and halving the associated memory traffic.

Additional optimizations: a precomputed reciprocal `inv_d = 1/d` replaces division with multiplication in the interior stencil; double-buffer pointer exchange of `data/n_data` eliminates a full-domain memory copy each iteration; neighbour-existence flags (`check_north`, etc.) are hoisted outside the triple loop to eliminate per-point sentinel comparisons.

1.3 Code Snippets

1. Communication–Computation Overlap (Core Pattern)

```

1 nreq = 0;
2 if(north != -1){
3     for(ll i=0; i<F; i++){
4         MPI_Isend(&data[i][x_len*val], 1, ns_type, north, i, COMM, &request[nreq++]);
5         MPI_Irecv(&data[i][0], 1, ns_type, north, i, COMM, &request[nreq++]);
6     }
7 }
8 // ... south, east, west, front, back follow identically
9
10 stencil_compute_core(data, n_data); // interior: no ghost dependency
11 MPI_Waitall(nreq, request, MPI_STATUSES_IGNORE);
12 stencil_compute_boundaries(data, n_data); // boundary: needs ghost data
13
14 double** temp = data; data = n_data; n_data = temp; // O(1) pointer swap

```

All halo sends and receives are issued prior to any computation. The interior stencil executes immediately, effectively hiding transfer latency behind productive arithmetic. Upon return of `MPI_Waitall`, boundary cells dependent on ghost data are computed, and the buffer pointers are exchanged.

2. Isovalue Crossing Detection (3-Direction Scheme)

```

1 for(11 j3=val; j3<nz+val; j3++){
2   for(11 j2=val; j2<ny+val; j2++){
3     for(11 j1=val; j1<nx+val; j1++){
4       double v1 = data[i][c];
5       if(check_front || (j3!=val))    // -z
6         sum1 += ((iso_val - v1) * (iso_val - data[i][c-plane]) <= 0);
7       if(check_north || (j2!=val))    // -y
8         sum1 += ((iso_val - v1) * (iso_val - data[i][c-x_len]) <= 0);
9       if(check_west || (j1!=val))     // -x
10        sum1 += ((iso_val - v1) * (iso_val - data[i][c-1]) <= 0);
11     }
12   }
13 }

```

Each cell inspects only the three negative-direction neighbours, ensuring every interior edge is counted exactly once. The product form is branchless and amenable to compiler vectorisation. At domain boundaries, hoisted boolean flags guard the check when no neighbouring process exists.

2 Code Compilation and Execution Instructions

Compilation The source is compiled using the MPI C compiler wrapper with the -O3 optimisation flag and linking the math library for mathematical operations:

```
1 mpicc -O3 -o stencil src.c
```

Here -O3 enables loop vectorisation and inlining.

Execution The program takes 12 positional arguments:

```
1 mpirun -np <P> ./stencil <d> <ppn> <px> <py> <pz> <nx> <ny> <nz> <T> <seed> <F> <isovalue>
```

For example, to run the $P=32$, 120^3 configuration:

```
1 mpirun -np 32 ./stencil 7 32 4 4 2 120 120 120 5 1000 2 500
```

SLURM Job Script. On the PARAM Rudra cluster, each configuration is submitted via a SLURM batch script. A representative example is provided below:

```

1 #!/bin/bash
2 #SBATCH --job-name=p32_nx120
3 #SBATCH --output=out_p32_nx120.txt
4 #SBATCH --ntasks-per-node=32
5 #SBATCH -N 1
6 #SBATCH --cpus-per-task=1
7 #SBATCH --time=00:10:00
8 #SBATCH --partition=cpu
9
10 module load compiler/oneapi-2024/mpi
11 mpicc -O3 -o stencil src.c -lm
12 for run in {1..5}; do
13   mpirun -np 32 ./src 7 32 4 4 2 120 120 120 5 1000 2 500
14 done

```

Each configuration is run 5 times for statistical reliability. The outputs are appended across runs, with each run producing T lines of isovalue counts followed by the wall-clock time.

Output Format. Rank 0 writes to the output file:

```

1 <count_field1> <count_field2>    # time step 1
2 ...
3 <count_field1> <count_field2>    # time step T
4 <time_in_seconds>

```

3 Results

Figure 1 presents the execution time (in seconds) across all 8 configurations, with each boxplot summarising five independent runs.

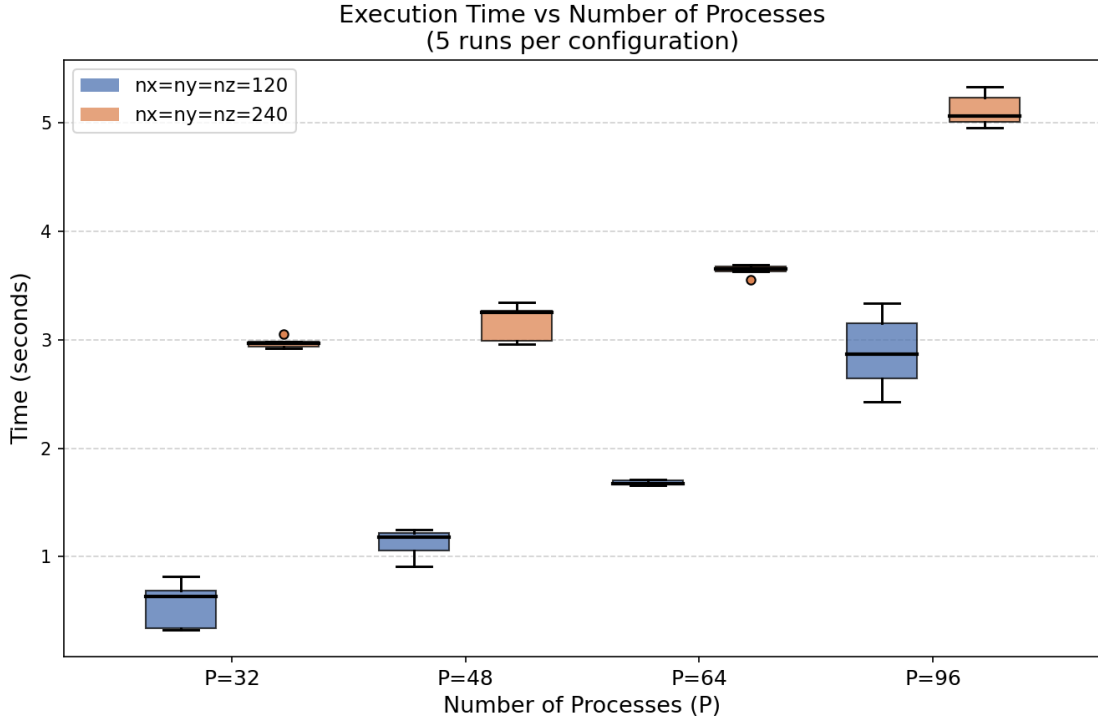


Figure 1: Execution time vs. number of processes for both sub-domain sizes. Each boxplot aggregates five runs per configuration.

Effect of sub-domain size. For every process count, the 240^3 configuration is approximately 4–5 $\times$ slower than the 120^3 configuration. Since each sub-domain grows by a factor of 8 in volume ($240^3/120^3 = 8$), a sub-linear increase in execution time indicates that the communication–computation overlap successfully amortises a portion of the halo transfer cost against the larger compute workload.

Effect of process count (weak scaling). The per-process sub-domain size is held constant, so increasing P raises the total problem size proportionally. Execution time increases with P due to growing communication volume and synchronisation overhead. A notable jump occurs between $P=48$ and $P=64$: here the job transitions from a single-node deployment (ppn = 48) to a two-node deployment (ppn = 32), introducing inter-node communication latency over the network interconnect rather than intra-node shared-memory transfers.

Variance. The 120^3 configurations at $P=96$ exhibit noticeably higher variance compared to other configurations. This is attributed to contention on the inter-node interconnect under higher process counts. The remaining configurations show tight distributions, indicating stable and reproducible performance.

4 Conclusions

1. **Communication overhead dominates scaling behaviour.** Since the sub-domain size per process is fixed, increasing the process count does not reduce per-process computation but does increase the aggregate halo exchange volume and global reduction cost. This explains the monotonic rise in execution time with P .
2. **Inter-node communication is the primary bottleneck.** The transition from single-node ($P=32, 48$) to multi-node ($P=64, 96$) configurations introduces a disproportionate increase in execution time, confirming that network latency and bandwidth across nodes are significantly more expensive than intra-node data movement.
3. **Communication–computation overlap is effective.** The 240^3 sub-domain, despite being 8 $\times$ larger in volume, incurs only a 4–5 $\times$ increase in wall time relative to 120^3 . The larger interior-to-boundary ratio at 240^3 allows the overlapped interior stencil to mask a greater fraction of halo transfer latency.
4. **MPI derived datatypes and the three-direction isocount contribute to overall efficiency.** Zero-copy halo transfers eliminate auxiliary buffer overhead, while restricting the isovalue sweep to three directions per cell reduces the per-point work by an order of magnitude compared to a full 26-neighbour traversal. Together with reciprocal multiplication and pointer swapping, these optimisations keep the per-iteration compute cost tight, ensuring that communication — not computation — remains the limiting factor at scale.